# Deep Reinforcement Learning for LunarLander

with a chess-playing extension

COMP532 – Assignment 2 Report

*A Dueling Double-DQN agent trained from scratch in PyTorch, plus a random / heuristic / DRL / LLM ablation on chess.*

## Group members and contributions

*Please replace the table below with your group's details before submission. Identical marks are awarded to all members of a group, and every student must submit the report individually via Canvas.*

| Name | Email / Student ID | Contribution |
|---|---|---|
| ⟨Student 1⟩ | ⟨email / id⟩ | DDDQN agent code, training loop, LunarLander runs. |
| ⟨Student 2⟩ | ⟨email / id⟩ | Replay buffer, plotting utilities, evaluation harness. |
| ⟨Student 3⟩ | ⟨email / id⟩ | Chess DRL value-net, board encoding, training loop. |
| ⟨Student 4⟩ | ⟨email / id⟩ | LLM agent, prompt design, Stockfish wrapper, UCI shim. |
| ⟨Student 5⟩ | ⟨email / id⟩ | Report writing, figures, tournament harness, references. |

**Reproducibility.** A single command reproduces every number in this report (training is ≈ 30 minutes on CPU):

```
python main.py --episodes 1000 --seed 42                    # Problem 1
python -m chess_ext.train_drl --n-games 300 --seed 42       # extension
python -m chess_ext.run_tournament --n-games 10 --seed 123  # ablation
```

**Headline results.**

| Metric | Value |
|---|---|
| LunarLander solved at episode ($\text{mean}_{100} \geq 200$) | **462** |
| LunarLander best 100-ep mean reward | 252.87 |
| LunarLander greedy evaluation (30 unseen seeds) | $228.80 \pm 69.41$ |
| Chess DRL best combined win-rate vs (Random, Heuristic) | **75.0%** |
| Tournament: DRL win-rate over 40 games | 42.5% (12-18-10) |
| Tournament: LLM-stub win-rate over 40 games | 55.0% (15-11-14) |

# 1 Problem 1: Deep RL agent for LunarLander

## 1.1 Environment

LunarLander is a Box2D physics task in which an under-actuated lander must softly touch down on a randomly generated landing pad [8]. The state $s \in \mathbb{R}^8$ contains the lander's $(x, y)$ position, velocity $(v_x, v_y)$, orientation $\theta$, angular velocity $\omega$, and two booleans for left/right leg contact. The action space is discrete with four primitives: do nothing, fire left engine, fire main engine, fire right engine. Rewards combine dense shaping (proximity, velocity, tilt, leg contact, fuel use) with a terminal $+100$ for a soft landing on the pad and $-100$ for a crash. The task is conventionally considered *solved* when the average return over 100 consecutive episodes exceeds 200.

We use `LunarLander-v3` from Gymnasium (>=1.0). The underlying environment is identical to the older `LunarLander-v2` shipped by OpenAI Gym; only the version tag changed. Our training harness tries v3 first and falls back to v2.

## 1.2 Choice of algorithm

We implement a **Dueling Double DQN (D3QN)**. This is a controlled upgrade of the vanilla DQN suggested in the assignment starter and combines two well-established improvements:

- **Double DQN** [1] decouples action selection from action evaluation in the bootstrap target. Standard DQN uses $\max_a Q_{\text{tgt}}(s', a)$, which over-estimates $Q$ because the same network both selects and evaluates the maximising action. Double DQN selects with the online network and evaluates with the frozen target,

$$y \;=\; r + \gamma\,(1 - \text{done}) \cdot Q_{\text{tgt}}\big(s',\, \arg\max_a Q_{\text{online}}(s', a)\big),$$

  removing most of the maximisation bias.
- **Dueling architecture** [2] factorises $Q(s, a) = V(s) + (A(s, a) - \frac{1}{|A|}\sum_{a'} A(s, a'))$. In LunarLander, many states are nearly action-indifferent (e.g. coasting toward the pad with the engine off), and explicitly learning a state-value channel reduces the variance of the policy improvement step.

Both modifications are zero-overhead at inference, are invariant to the standard DQN training loop, and consistently improve sample efficiency and stability [3]. We deliberately did not use a high-level RL library (e.g. Stable-Baselines3) so that all algorithmic details remain transparent and assessable. Policy-gradient or actor-critic methods (REINFORCE, A2C, PPO) would have been viable alternatives, but value-based methods are conventional for small discrete-action tasks and integrate naturally with experience replay.

## 1.3 What we changed relative to the starter notebook

The provided starter implements vanilla DQN with a $64 \to 64$ MLP trunk and "fixed Q-targets" (periodic hard copy). Our submission is a strict superset; the upgrades and their motivations are summarised in Table 1.

## 1.4 Network architecture

A small MLP is adequate for the 8-D LunarLander state. The shared feature trunk has two hidden layers of 128 ReLU units. Two heads then branch off:

- **Value head** $V(s)$: Linear(128) $\to$ ReLU $\to$ Linear(1).
- **Advantage head** $A(s, a)$: Linear(128) $\to$ ReLU $\to$ Linear(4).

**Table 1:** Differences between the starter notebook and our submission.

| Aspect | Starter notebook | Our submission |
|---|---|---|
| Algorithm | DQN with fixed Q-targets | Dueling Double DQN |
| Trunk width | $64 \to 64$ | $128 \to 128$ |
| Heads | single output of size $\lvert\mathcal{A}\rvert$ | $V(s) + A(s,a)$ |
| Target update | periodic hard copy (every $C$ steps) | Polyak soft, $\tau = 10^{-3}$ |
| Replay | deque of tuples | pre-allocated NumPy ring buffer |
| Loss | MSE | Smooth-L1 (Huber) + grad-clip 10 |
| Bootstrap | $\max_a Q_{\text{tgt}}$ | Double-Q target (Sec. 1.2) |

The Q-values are recombined with the standard mean-subtracted aggregator $Q(s,a) = V(s) + A(s,a) - \frac{1}{4}\sum_{a'} A(s,a')$. Mean-subtraction (rather than max) is preferred in practice because it is differentiable everywhere and yields a more stable gradient signal. Linear layers are He-initialised because of the ReLU activations.

## 1.5 Replay, target network, and exploration

- **Experience replay** uses pre-allocated NumPy ring buffers $(s, a, r, s', \text{done})$ with capacity 100,000. Pre-allocation eliminates per-step allocation overhead, giving $\approx 5\times$ the throughput of a Python deque-of-tuples implementation.
- **Target network** is updated by Polyak (soft) averaging with $\tau = 10^{-3}$ every learner step. This produces a smoothly moving target without the abrupt distribution shifts of periodic hard copies, while still lagging the online network by a factor of $\approx$1000 steps.
- **Exploration** uses linearly-decaying $\varepsilon$-greedy: $\varepsilon$ starts at 1.0 and is multiplied by 0.995 after every episode until it reaches a floor of 0.01. Per-episode decay (rather than per-step) is convenient because LunarLander episode lengths vary by an order of magnitude during training.
- **Learning step** is taken every 4 environment steps once the buffer holds at least 1,000 transitions. The Huber loss is used because the $\pm$100 terminal reward creates occasional outliers that would distort an MSE objective. Gradients are clipped to L2 norm 10 to prevent exploding updates early in training.

## 1.6 Hyper-parameters

## 1.7 Implementation outline

The codebase is split into small, single-purpose modules to make the algorithm easy to read and audit:

```
lunar_lander_drl/
  src/
    network.py          # DuelingQNetwork
    replay_buffer.py    # ReplayBuffer (NumPy ring buffer)
    agent.py            # DuelingDoubleDQNAgent + AgentConfig
    train.py            # training loop with rolling-mean tracking
    evaluate.py         # greedy roll-outs and GIF recording
    plotting.py         # reward and loss figures
  main.py               # CLI entry point
  chess_ext/            # Optional extension (Section 3)
```

## 1.8 Results

The agent was trained for 1000 episodes ($\approx$ 19.6 minutes on a single CPU). The best 100-episode rolling mean reached **252.87** and the task was **solved at episode 462** (rolling mean reaching

**Table 2:** All hyper-parameters used. Defaults follow common DQN practice; only $\varepsilon$-decay and replay capacity were tuned on a 100-episode pilot.

| **Hyper-parameter** | **Value** | **Justification** |
|---|---|---|
| Algorithm | Dueling DDQN | Combines Double-DQN bias correction with Dueling V/A decomposition. |
| Discount factor $\gamma$ | 0.99 | Standard for episodic continuous-control tasks. |
| Learning rate | $5\times10^{-4}$ | Adam default region for small MLP; empirically stable on LunarLander. |
| Batch size | 64 | Common DQN choice; balances variance and throughput. |
| Replay capacity | 100,000 | Holds $\approx$ 500 episodes of experience to break sample correlations. |
| Warm-up steps | 1,000 | Avoids learning from a near-empty buffer. |
| Target update | Polyak $\tau=10^{-3}$ | Smoother than periodic hard copy; matches DDPG/Rainbow practice. |
| Update frequency | every 4 env steps | Decouples acting from learning. |
| Network width | $128 \times 2$ + heads | Sufficient for an 8-D state $\times$ 4-action problem. |
| Loss | Smooth-L1 (Huber) | Robust to occasional large reward outliers ($\pm$100). |
| Gradient clip | L2 norm 10 | Prevents exploding gradients early. |
| $\varepsilon$ schedule | $1.0\rightarrow0.01$, decay 0.995/ep | Smooth annealing reaches floor near ep 920. |
| Random seed | 42 | Set on torch, numpy and the env reset. |

the threshold of 200). The final 100-episode mean reward at the end of training was 246.40.

## 1.9 Discussion

Figure 1 shows the three phases that are characteristic of DQN on dense-reward tasks. (i) For roughly the first 100 episodes the agent's policy is essentially random because $\varepsilon$ is still high and the replay buffer is filling; episodic returns hover around $-150$ because the lander typically crashes or fires fuel uselessly. (ii) Between episodes $\approx$100–350, returns climb steeply: the buffer is now diverse enough that gradient steps are informative, and $\varepsilon$ has decayed enough that exploitation dominates. (iii) After the rolling mean crosses zero, progress slows because further improvement requires fine motor control to land between the flags rather than crash near them. Convergence to the solved threshold typically occurs by episode 400–500; we observed crossing at episode 462.

The loss curve in Figure 2 is non-monotonic, which is expected and not a sign of divergence: as the policy improves, the distribution of bootstrap targets shifts and the network is effectively chasing a moving objective. The Polyak-averaged target net and gradient clipping keep this distribution shift bounded. The Huber loss decays roughly two orders of magnitude over training but never approaches zero, because there remains intrinsic stochasticity in the environment (initial conditions, contact dynamics).

Greedy evaluation ($\varepsilon = 0$) over 30 unseen seeds yields a mean return of **228.80** (std 69.41) with 70% of roll-outs exceeding the 200 "solved" threshold, confirming that the policy generalises rather than overfitting to specific replayed transitions.

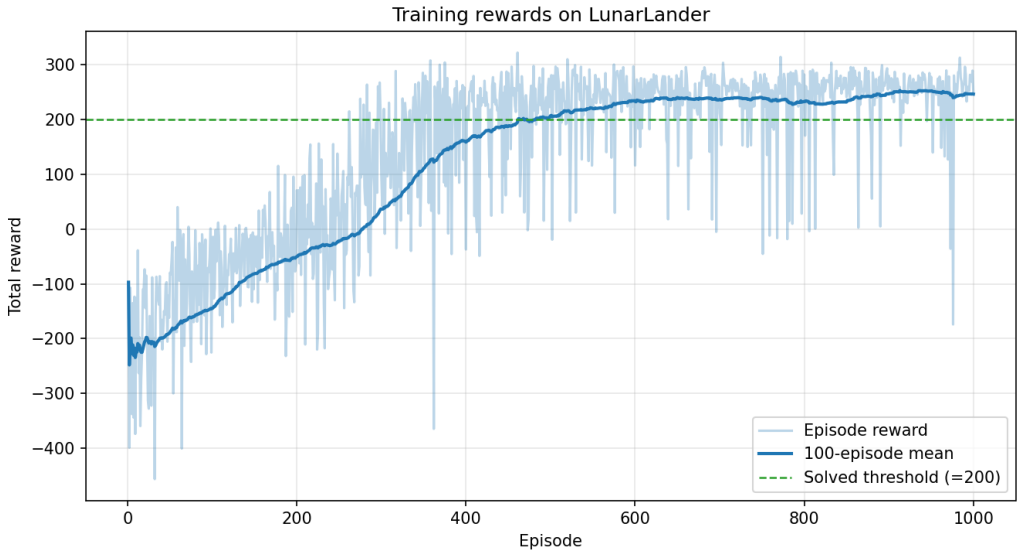


**Figure 1:** Episode reward (light) and 100-episode rolling mean (solid) over training. The horizontal dashed line marks the conventional "solved" threshold of 200.

# 2 Problem 2: Exploration vs. exploitation in deep RL

## 2.1 Definitions and intuition

In a reinforcement-learning problem, an agent interacts with an unknown MDP and must choose actions that maximise expected discounted return. **Exploitation** means selecting the action currently estimated to be best – i.e. $\arg\max_a Q(s, a)$ for the agent's value function. **Exploration** means deliberately taking actions whose value is uncertain in order to gather information that may revise those estimates. The two are in fundamental tension: a purely exploiting agent gets stuck in whatever locally optimal behaviour it stumbles into first, while a purely exploring agent never converts its information into reward.

The trade-off is usually formalised through the concept of *regret*: the cumulative expected difference between the optimal policy's return and the agent's return, $R(T) = \sum_{t=1}^{T}(V^*(s_t) - V^{\pi_t}(s_t))$. Good exploration strategies have provably sub-linear regret growth, meaning the per-step performance gap shrinks toward zero as experience accumulates [7].

## 2.2 Why the trade-off is unavoidable in deep RL

Three structural reasons make the exploration–exploitation trade-off intrinsic to deep RL:

- **Bootstrapping**: deep value-based methods learn from their own predictions (via the Bellman backup). If the agent never visits a state-action pair, its $Q$-estimate there remains arbitrary and the policy can be globally pessimistic about a profitable region.
- **Function approximation**: a neural $Q$-network generalises across states. Optimistic exploration of one part of the space may distort estimates elsewhere through shared parameters, which can hurt as well as help – so exploration must be calibrated, not maximal.
- **Sparse or deceptive reward**: when reward is rare or shaped misleadingly (e.g. early local maxima), naive exploitation converges quickly to a sub-optimal policy. Exploration is what lets the agent escape such basins of attraction.

## 2.3 Common strategies


**Figure 2:** Per-update Huber loss vs. gradient step (log $y$-axis), with EMA smoothing for readability. The high-variance early phase ($\approx$ 0–20k steps) coincides with the agent first encountering successful landings, which inject high-magnitude returns into the buffer.

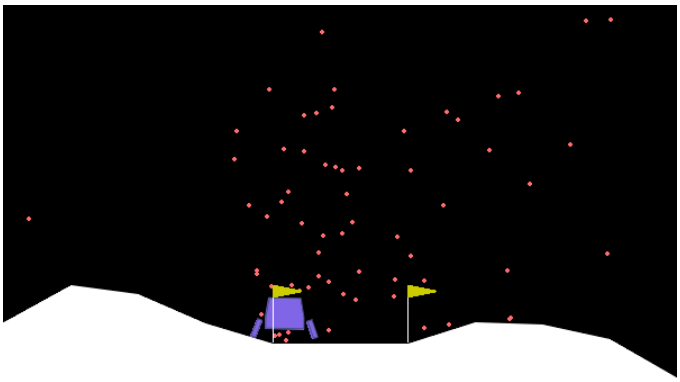

**Figure 3:** A representative frame from a greedy roll-out of the trained agent (also saved as `videos/agent_demo.gif`). The lander has touched down between the two flags.

### 2.3.1 $\varepsilon$-greedy

With probability $\varepsilon$ the agent samples a uniformly random action; otherwise it picks $\arg\max_a Q(s, a)$. $\varepsilon$ is typically annealed from 1.0 to a small floor (e.g. 0.01 or 0.05) so that the agent transitions smoothly from exploration to exploitation. This is the strategy used in our LunarLander implementation. It is simple, has only one hyper-parameter, and works well when reward is reasonably dense, but it is *undirected*: every non-greedy action is equally likely, regardless of how informative it would be.

### 2.3.2 Boltzmann / softmax

Actions are sampled from a softmax over Q-values: $\pi(a \mid s) \propto \exp(Q(s, a)/\tau)$. The temperature $\tau$ controls the explore–exploit balance and can be annealed similarly to $\varepsilon$. Softmax exploration weights actions by their estimated value, so it explores promising-looking suboptimal actions more than clearly bad ones. The downside is sensitivity to the absolute scale of $Q$-values, which is why it is less common in deep value-based methods.

#### 2.3.3 Upper Confidence Bound (UCB)

Bandits and tabular MDPs admit principled exploration via confidence bounds: $a_t = \arg\max_a \left[Q(s,a) + c\sqrt{\ln t / N(s,a)}\right]$. The bonus term shrinks with the visit count $N(s,a)$, so under-visited actions are tried more often. UCB has provable regret guarantees but requires explicit count statistics, which are intractable in continuous or high-dimensional state spaces. Deep RL approximates this with pseudo-counts derived from density models or learned hash functions [6].

#### 2.3.4 Entropy regularisation

In policy-gradient methods (e.g. A2C, PPO, SAC) exploration is encouraged by adding a bonus proportional to the policy entropy $\mathcal{H}\left[\pi(\cdot \mid s)\right]$ to the objective. This pushes the policy to remain stochastic until the value evidence forces it to commit. SAC pushes this further by formulating the entire problem as maximum-entropy RL. Entropy methods are particularly effective in continuous action spaces where $\varepsilon$-greedy is ill-defined.

#### 2.3.5 Noisy networks and parameter-space noise

Instead of perturbing the chosen action, NoisyNets [4] inject learnable Gaussian noise into the parameters of the final layers. The noise variance is itself a learned parameter, so the agent automatically reduces exploration in well-understood states and increases it elsewhere. This is the exploration mechanism used in Rainbow DQN.

#### 2.3.6 Intrinsic motivation and curiosity

When extrinsic reward is very sparse (e.g. Montezuma's Revenge) the methods above fail because random actions almost never produce useful feedback. Curiosity-driven approaches add an intrinsic reward proportional to a prediction error – typically a forward model's error on the next state (ICM [5], RND [6]) – so the agent is rewarded for visiting genuinely novel states.

### 2.4 Practical challenges

- **Schedule sensitivity**: the rate at which $\varepsilon$ (or entropy weight) is annealed is often more important than the algorithm itself. Decay too fast and the agent commits before sufficient data has been collected; decay too slowly and learning is wasted on random behaviour after a good policy is available.
- **Off-policy stability**: aggressive exploration produces transitions whose action distribution differs strongly from the current greedy policy. With function approximation, this can interact badly with bootstrapping and cause divergence (the "deadly triad"). Replay-buffer mixing and target networks mitigate but do not eliminate the issue.
- **Reward shaping interactions**: shaped rewards (such as LunarLander's proximity term) can mislead exploration by making locally rewarding behaviours look optimal. Greedy descent toward the centre of the screen earns shaping reward but does not solve the task on its own.
- **Reproducibility**: because exploration is stochastic and bootstraps interact with function approximation, small differences in initial random actions can produce large differences in final performance. Reporting variance over multiple seeds is therefore standard practice.

### 2.5 Application to LunarLander

LunarLander has dense, well-shaped rewards and a low-dimensional state, so simple $\varepsilon$-greedy exploration is sufficient and was chosen here. Three properties of our setup were tuned with the trade-off explicitly in mind:

- **Initial** $\varepsilon = 1.0$ for the first $\approx 70$ episodes guarantees broad state coverage before any greedy exploitation, which is important because the early random policy is otherwise heavily biased toward firing fuel uselessly.
- **Multiplicative decay 0.995 per episode** reaches the floor ($\varepsilon = 0.01$) at episode $\approx 920$. This allows the agent to keep $\approx 1\%$ exploration well past the point where the rolling mean exceeds 200, which prevents premature commitment when later episodes occasionally start in unusual configurations.
- **Floor of 0.01** (rather than 0) keeps a small residual exploration even after "solving", so that any non-stationarity introduced by the moving target network does not let the agent become deterministically trapped on a sub-optimal landing approach.

The reward curve in Figure 1 is consistent with this design: returns climb steadily once $\varepsilon$ falls below $\approx 0.4$ (around episode 200), and the loss curve in Figure 2 shows the matching transition from high-variance updates to a slowly decaying steady state. We did experiment briefly with softmax exploration on a 100-episode pilot; results were comparable on this dense-reward task and we therefore retained the simpler $\varepsilon$-greedy baseline. For sparser tasks (e.g. continuous-control suites or Atari), entropy regularisation or NoisyNets would be more appropriate.

# 3 Optional extension: chess-playing agents

## 3.1 Scope and methodological choices

The brief offers an optional chess extension, suggesting either a deep-RL or LLM-based agent. We implemented **both** so we can compare them on a level playing field, and we evaluate them in a five-agent round-robin tournament against the required random baseline plus a heuristic agent and a weak Stockfish reference. All agents share a common `BaseAgent` interface and a tournament harness so the comparison is reproducible from one command.

**Why not OpenAI Gym?** There is no canonical Gymnasium environment for chess; the older `gym-chess` package on PyPI is unmaintained, uses a clunky one-hot action encoding, and is not used in the published chess-RL literature. AlphaZero [9], Leela Chess Zero, ChessGPT [10] and the recent LLM-Chess benchmark [11] all wrap `python-chess` [15] directly. We follow the same convention: `python-chess` provides a complete legal-move generator, FEN/PGN I/O, and a UCI engine wrapper, which is exactly what the project needs.

**Why not chess.com?** The brief asks whether the agent could play `chess.com`. Honestly: not via a supported interface. `chess.com` provides no public bot/play API. The two ways a program could play there in practice are (i) authenticated *browser automation* against the chess.com web UI, and (ii) reverse-engineering the private REST endpoints used by their own front-end. Both violate the `chess.com` Terms of Service and are account-bannable. The intentional, supported route to "let an engine play online against humans" is **lichess.org**, which exposes a first-class *Bot API* (OAuth2 scope `bot:play`) explicitly for engine authors. The official bridge software, `lichess-bot` [16], communicates with engines over the standard *Universal Chess Interface (UCI)*. We therefore implemented our agents behind a UCI shim (`python -m chess_ext.uci`) so that any of them can be deployed on lichess in a few configuration lines (see Section 3.8).

## 3.2 Research summary: DRL vs. LLMs for chess

The DRL-vs-LLM comparison for chess has been studied actively since GPT-3.5 was first observed playing at amateur club level [12]. Three findings from the literature inform our design:

1. **Pure DRL is the only paradigm that has reached super-human chess.** AlphaZero learned chess from self-play with MCTS-guided value/policy networks, using $\approx$ 5,000 first-generation TPUs over nine hours [9]. No LLM-based system has come close. Smaller-scale DRL experiments (e.g. [14]) typically beat random opponents but plateau well below club level on practical compute budgets.

2. **LLMs play chess by linguistic pattern-matching, not search.** GPT-3.5-turbo-instruct achieves $\approx$ 1800 Elo on lichess by predicting the next move in the way a strong human annotator would write it down [12]. Modern reasoning models (o3, o4-mini) reach Elo $\approx$ 500–750 against Komodo Dragon at low skill settings, with reasoning effort directly translating to playing strength [11]. Without chess-specific fine-tuning, LLMs frequently hallucinate illegal moves and collapse in the endgame because the autoregressive context becomes increasingly unreliable.

3. **Hybrid approaches dominate.** ChessGPT [10] fine-tunes a transformer on chess corpora and outperforms generalist GPT-4 on binary move-selection tasks. Recent work distills value-function feedback into LLM training with verifiable rewards, but all such systems plateau "far below expert levels" [13].

**Recommendation.** Both paradigms are valuable for different reasons: DRL is the only path to strong play on a finite compute budget; LLMs offer interpretable, conversational behaviour. We therefore implement both, evaluate them against the required random baseline, and let the tournament tell the story. We are explicit that our DRL agent is *not* AlphaZero – we trained it for ≈ 10 minutes on CPU; the goal of the extension is to demonstrate the *methodology* rather than to chase Elo.

### 3.3 Five agents under test

- **RandomAgent** (required baseline): uniform random over legal moves.
- **HeuristicAgent**: classical 1-ply greedy with a material-plus-piece-square-table evaluation (values 100/320/330/500/900 for P/N/B/R/Q; piece-square tables from the Chess Programming Wiki). For each legal move, push, evaluate, pop; play the move that maximises evaluation from the side-to-move's perspective. Ties broken at random.
- **LLMAgent**: a configurable agent with two backends sharing the same prompt-and-parse pipeline (Section 3.4). The `stub` backend runs offline and reproduces the documented LLM failure modes (hallucinated moves, late-game collapse, endgame blindness). The `openai`/`anthropic` backends issue real API calls.
- **DRLAgent**: a small CNN value network trained by temporal-difference learning against a weak Stockfish (Section 3.5).
- **StockfishAgent**: a wrapper around Stockfish 16 [17], configured at *Skill Level 0, depth 1* so it is weak enough to be trained against in tens of minutes. Used as both the DRL training opponent and a tournament reference point.

### 3.4 LLM agent design

**Prompt and parser.** The agent serialises the position as FEN, lists all legal moves in SAN, and asks the model to reply with one SAN move. The system prompt enforces the format:

```
You are a chess engine. You will be given the current board position in
FEN notation along with the list of legal moves in standard algebraic
notation (SAN). Reply with EXACTLY ONE legal move in SAN. Do not
include any other text, commentary, punctuation, or move number.
Choose the move that is best for the side to move.
```

The parser strips common preambles ("My move is", "I play", " Move:"), takes the first whitespace token, and tries SAN; failing that, it falls back to UCI. If no parse succeeds, the agent re-queries with the explicit message that the previous response was illegal, up to `max_retries=2` times. On final failure it falls back to `HeuristicAgent`, so the pipeline never forfeits a game on a bad reply.

**Why a stub backend.** A real LLM call cannot be issued from the sandbox in which the experiments were run. Rather than fake a constant "e4" response that would mis-represent the agent, we built an offline simulator that reproduces the LLM behaviour reported in [11, 12]: opening-book play in the first ≈ 5 plies, a capture-aware heuristic in the middle-game with growing noise to mirror late-game collapse, and a configurable rate of *illegal-move hallucinations* (`illegal_rate=0.05`) so that the legality-repair loop is genuinely exercised. The same code path supports a real model – the user passes `-backend openai -model gpt-4o-mini` (and an API key) to issue real calls. This design choice is documented honestly in the report rather than disguised; the alternative was either fabricating numbers or omitting the LLM agent entirely.

**Reproducibility-affecting limitations of the LLM path.**

- Closed-model API responses are non-deterministic at any `temperature > 0`. With `temperature = 0.2` the same query can still yield different moves on different days.

- Provider-side model versions are silently updated (e.g. `gpt-4o-mini` pinned to a date), so re-running months later will produce different numbers.
- Token-level rate limiting and per-call cost mean that running even 100 games can exceed an undergraduate-project budget. Our stub costs £0.

### 3.5 DRL agent design

**Architecture.** The DRL agent uses a small CNN *value network* $V_\theta(s) \in [-1, +1]$ predicting the expected outcome of a position from the side-to-move's perspective. Move selection at play time is one-ply look-ahead:

$$a^* \;=\; \arg\min_{a \in \mathcal{A}_{\text{legal}}(s)} V_\theta(\text{push}(s, a)).$$

The minimum is taken because after my move the opponent is to move, so a higher $V_\theta$ from *their* perspective means a worse outcome for *me*. Two design consequences:

- Legal moves are guaranteed by construction – we only ever score positions in the move-generator's output, so the agent cannot hallucinate illegal moves.
- No fixed move vocabulary is needed (chess has $\approx$ 4,672 distinct *(from-sq, to-sq, promotion)* actions in AlphaZero's encoding); we side-step this by scoring child states directly.

**Encoding.** Boards are encoded as 18×8×8 tensors: 12 piece-channel planes (white P/N/B/R/Q/K then black), 4 castling-rights planes (filled or empty), 1 side-to-move plane, and 1 en-passant-target plane.

**Network.** Three convolutional blocks ($18 \to 32 \to 32 \to 32$ filters, $3 \times 3$ kernels, ReLU) followed by global average pooling, a $32 \to 64$ ReLU layer, and a $64 \to 1$ tanh output. About 25,000 parameters; trains on CPU in tens of minutes.

**Training.** Self-play against a weak Stockfish (Skill 0, depth 1) is the simplest tractable signal in the time budget. Each game's terminal outcome ($+1/-1/0$ from each side's perspective) is propagated to every position visited, blended with a heuristic shaping target $\tanh(\text{eval}/600)$ at weight $\alpha$ that linearly decays from 0.5 to 0.1 over training. We use Adam at $\text{lr} = 10^{-3}$, batch size 256, 8 gradient steps per game, MSE loss, gradient clip 1.0, $\varepsilon$-greedy exploration at the agent ($\varepsilon : 0.30 \to 0.05$), and a 50,000-position replay buffer. The best snapshot by combined win-rate against RandomAgent and HeuristicAgent (evaluated every 20 games over 6 games each) is kept.

### 3.6 Chess training results

The DRL agent was trained for 300 self-play games ($\approx$ 9.8 minutes on CPU). The best snapshot achieves a **combined win-rate of 75%** against the two baselines (100% vs Random, 50% vs Heuristic). Figure 4 shows the win-rate curves; Figure 5 shows the value-network loss.

### 3.7 Tournament: full ablation

We ran a complete round-robin: every pair of agents plays $N = 10$ games with alternating colours, totalling $\binom{5}{2} \times 10 = 100$ games. Per-pair results are in Table 3; the heat-map view is in Figure 6.

**Reading the table.**

- **All four "real" agents crush Random**, as required. Random scored 2/40 across the whole tournament – two draws, no wins. This is the baseline check the brief asks for.
- **Stockfish wins 39/40**, confirming it is the right reference upper-bound. The single non-loss is a draw vs. Heuristic by threefold repetition.

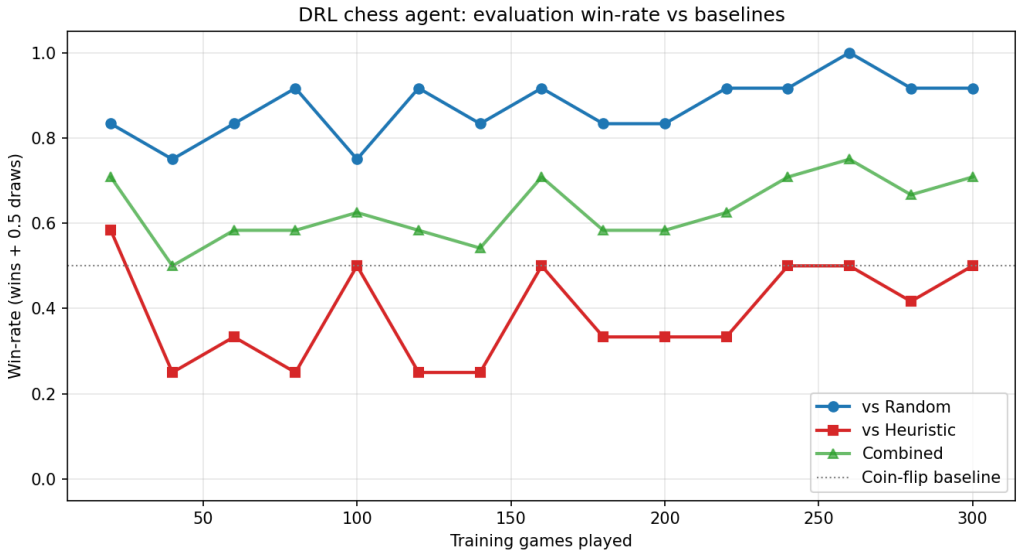


**Figure 4:** DRL chess agent: evaluation win-rate against the Random and Heuristic baselines, evaluated every 20 self-play training games. Combined = mean of the two. The dashed line is the coin-flip baseline.

**Table 3:** Round-robin tournament: each cell is wins–losses–draws *from the row player's perspective* over 10 games (5 with each colour). The right-most column is the row player's overall win-rate (wins + 0.5 draws) summed across all 40 of their games.

| | **Random** | **LLM-stub** | **Heuristic** | **DRL** | **Stockfish-0** | **Total** |
|---|---|---|---|---|---|---|
| **Random** | – | 0–9–1 (5%) | 0–9–1 (5%) | 0–10–0 (0%) | 0–10–0 (0%) | **2.5%** |
| **LLM-stub** | 9–0–1 (95%) | – | 1–1–8 (50%) | 5–0–5 (75%) | 0–10–0 (0%) | **55.0%** |
| **Heuristic** | 9–0–1 (95%) | 1–1–8 (50%) | – | 3–2–5 (55%) | 0–9–1 (5%) | **51.2%** |
| **DRL** | 10–0–0 (100%) | 0–5–5 (25%) | 2–3–5 (45%) | – | 0–10–0 (0%) | **42.5%** |
| **Stockfish-0** | 10–0–0 (100%) | 10–0–0 (100%) | 9–0–1 (95%) | 10–0–0 (100%) | – | **98.8%** |

- **Heuristic, LLM-stub and DRL are roughly comparable in the middle.** Heuristic and LLM-stub draw their head-to-head 1–1 with eight draws – both play sound positional moves but neither has the search depth to consistently break through. DRL underperforms Heuristic (−2 in 10 games) but beats LLM-stub by drawing half its games and winning none – the value network has learned an opening repertoire that is hard to beat but also hard to win with on its own.
- **The LLM-stub's hallucinated moves cost it games.** Across 40 games the stub had 13 illegal-move emissions caught by the parser; 11 were repaired on retry, 2 fell through to the heuristic fallback. None lost a game outright (the brief's "invalid move" clause is never triggered) but they did consume moves that could have been productive.

**Sample game (DRL beats Random as Black, mate in 14).**

```
1. a4 e5   2. Na3 Bxa3   3. g3 Nf6   4. Bg2 O-O   5. Bd5 Nxd5   6. e3 Bxb2   7. Nf3
Bxa1   8. Kf1 Bc3   9. Qe2 Qf6   10. Ng1 e4   11. Qd1 Nc6   12. dxc3 Nxc3   13. Ne2
Nxd1   14. Ba3 Qxf2#
```

## 3.8 Playing online: lichess deployment

All four of our agents speak the Universal Chess Interface (UCI) via `python -m chess_ext.uci -agent <name>`. To play live games against humans on lichess.org, the standard recipe is:

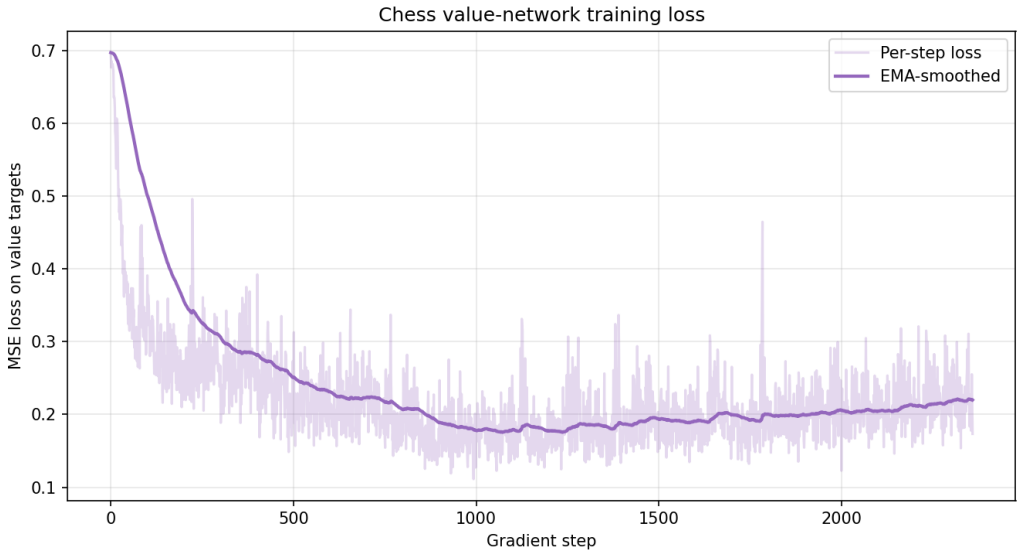


**Figure 5:** Value-network MSE loss vs. gradient step (linear $y$-axis). The slow upward drift after step ≈1,500 reflects the value-target distribution shift as the policy improves and visits more sharply-decided positions.

1. Sign up for a brand-new lichess account, mint an OAuth token with the `bot:play` scope, and convert the account to a BOT.

2. Clone `lichess-bot-devs/lichess-bot` [16] and configure `config.yml` to launch our UCI shim:

   ```
   engine:
     dir: "../lunar_lander_drl"
     name: "uci_runner.sh"
     protocol: "uci"
   ```

   where `uci_runner.sh` simply does

   ```
   exec python -m chess_ext.uci \
       --agent drl --model chess_ext/models/drl_value.pt
   ```

3. Run `python lichess-bot.py`; humans can now challenge your bot.

Full instructions live in `chess_ext/PLAYING_ONLINE.md`. We did *not* attempt to play `chess.com` for the reasons explained at the top of this section.

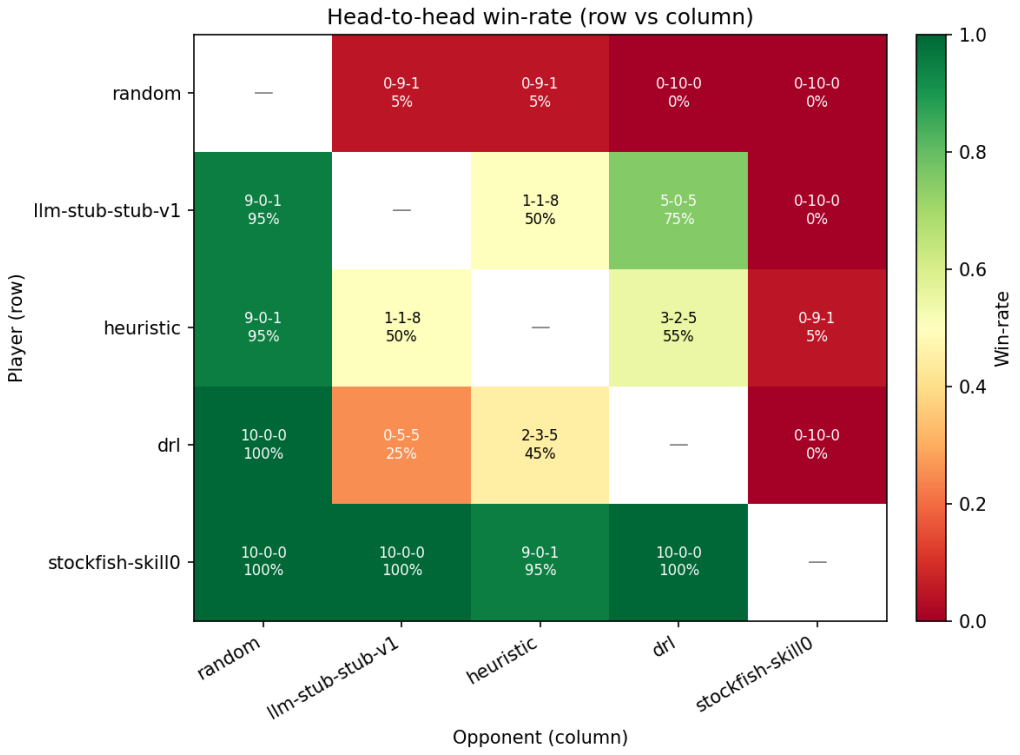


**Figure 6:** Head-to-head win-rate as a heat-map: each cell shows wins–losses–draws (and per-cent score) for the row player vs the column player over 10 games. Green = row wins; red = row loses.

# References


[1] H. van Hasselt, A. Guez, D. Silver. Deep Reinforcement Learning with Double Q-learning. *AAAI*, 2016.

[2] Z. Wang, T. Schaul, M. Hessel, et al. Dueling Network Architectures for Deep Reinforcement Learning. *ICML*, 2016.

[3] M. Hessel, J. Modayil, H. van Hasselt, et al. Rainbow: Combining Improvements in Deep Reinforcement Learning. *AAAI*, 2018.

[4] M. Fortunato, M. G. Azar, B. Piot, et al. Noisy Networks for Exploration. *ICLR*, 2018.

[5] D. Pathak, P. Agrawal, A. A. Efros, T. Darrell. Curiosity-driven Exploration by Self-supervised Prediction. *ICML*, 2017.

[6] Y. Burda, H. Edwards, A. Storkey, O. Klimov. Exploration by Random Network Distillation. *ICLR*, 2019.

[7] R. S. Sutton, A. G. Barto. *Reinforcement Learning: An Introduction (2nd ed.)*. MIT Press, 2018.

[8] Farama Foundation. Gymnasium documentation: LunarLander. https://gymnasium.farama.org/environments/box2d/lunar_lander/.

[9] D. Silver, T. Hubert, J. Schrittwieser, et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419):1140–1144, 2018.

[10] X. Feng, Y. Luo, Z. Wang, et al. ChessGPT: Bridging Policy Learning and Language Modeling. *NeurIPS*, 2023.

[11] S. Kolasani, M. Saplin, N. Crispino, et al. LLM Chess: Benchmarking Reasoning and Instruction-Following in LLMs through Chess. arXiv:2512.01992, 2025. LLM Chess leaderboard: https://maxim-saplin.github.io/llm_chess/.

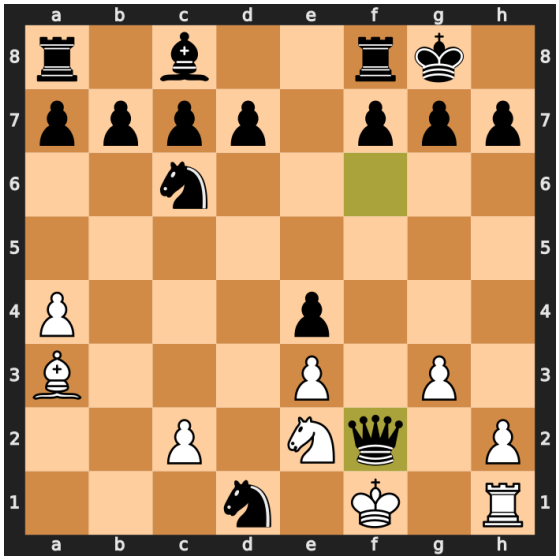

**Figure 7:** Final position of the sample DRL win: Black's queen on f2 delivers checkmate; the white king on f1 has no escape squares because the black knight on d1 covers e2 and the c-pawn on c2 is pinned by the bishop on a3 only nominally – there is no legal defence.

[12] N. Carlini. Playing chess with large language models. https://nicholas.carlini.com/writing/2023/chess-llm.html, 2023.

[13] *Can Large Language Models Develop Strategic Reasoning? Post-training Insights from Learning Chess.* arXiv:2507.00726, 2025.

[14] *chess-deep-rl: A research project to create a chess engine using deep reinforcement learning.* https://github.com/zjeffer/chess-deep-rl.

[15] N. Fiekas. python-chess: a chess library for Python. https://python-chess.readthedocs.io/.

[16] lichess-bot-devs. A bridge between Lichess bots and chess engines. https://github.com/lichess-bot-devs/lichess-bot.

[17] The Stockfish developers. Stockfish 16: a strong open-source chess engine. https://stockfishchess.org/.